Лекция 6 Контейнеризация и управление средой

Методические материалы курса · учебный год 2026–2027

Курс: Управление процессами машинного обучения MLOps Продолжительность: 2 академических часа
Связанная работа: ЛР6 «Контейнеризация и управление средой»

Цель лекции

Показать контейнер как проверяемую границу поставки ML-сервиса, а не как архив исходного кода. После занятия студент должен различать image и container, build-time и runtime, объяснять layers и cache, создавать минимальный non-root runtime, настраивать readiness и связывать image с версией кода и модели.

План занятия

Время	Содержание	Действие преподавателя
0–10 мин	Зачем ML-системе контейнер	Сопоставить «работает у меня» и переносимый контракт
10–25 мин	Image, container, registry и digest	Разобрать неизменяемый артефакт и экземпляр процесса
25–40 мин	Build context, layers и cache	Показать влияние порядка COPY и .dockerignore
40–55 мин	Многостадийная сборка	Отделить quality/build от runtime
55–68 мин	Безопасность runtime	Разобрать USER, read-only, capabilities и secrets
68–80 мин	Health, readiness и Compose	Показать контейнер без готовой модели
80–90 мин	Диагностика и ЛР6	Связать build log, inspect и smoke test

1 Контейнер как контракт поставки

Начните с вопроса: «Если requirements и исходный код находятся в Git, зачем нужен image?» Репозиторий описывает материалы для сборки, но не фиксирует итоговую файловую систему, системные библиотеки, пользователя процесса, команду запуска и готовность сервиса.

Контейнеризация должна обеспечить:

- одинаковый способ запуска на машине студента и runner;
- известную версию runtime и зависимостей;
- ограниченную поверхность исполнения;
- наблюдаемую готовность модели;
- прослеживаемость от commit до image ID или digest.

Контейнер не исправляет невоспроизводимые данные, утечку или плохие тесты. Он воспроизводимо упаковывает и хорошие, и плохие решения.

2 Image и container

Image — неизменяемый шаблон файловой системы и конфигурации. Container — запущенный экземпляр image с собственным состоянием процесса и writable layer. Один image может породить несколько containers.

Понятие	Вопрос	Пример
Dockerfile	как собрать?	инструкции FROM, COPY, RUN, USER
Image	что поставляем?	приложение, зависимости, модель, metadata
Container	что сейчас выполняется?	PID, сеть, health и runtime environment
Registry	где хранится image?	репозиторий образов
Tag	удобное имя ссылки	lr6-a1b2c3d
Digest	content-addressed идентификатор	sha256:...

Tag можно переназначить, digest зависит от содержимого. Поэтому latest не доказывает, какая версия развёрнута. В ЛР6 используется tag с short SHA и локальный image ID; к итоговому release добавится registry digest.

3 Build context

Build context передаётся builder и определяет, что доступно COPY. Если context содержит .git, данные, секреты и отчёты, они могут увеличить сборку или попасть в image layer.

.dockerignore исключает файлы до передачи. Он полезен для производительности и снижения риска, но не является хранилищем секретов. Если секрет уже попал в опубликованный image, его нужно отозвать, а не только дописать правило ignore.

Попросите аудиторию классифицировать каталоги starter-project:

- src, tests, data нужны quality-стадии;
- models не копируется, потому что учебная модель строится внутри build;
- .git, .env, reports, mlruns и документация не нужны runtime.

4 Layers и cache

Каждая инструкция создаёт слой или меняет конфигурацию image. Cache повторно использует слой, если инструкция и её входы не изменились. Поэтому стабильные зависимости копируют и устанавливают раньше часто меняющегося кода.

Плохой порядок:

```
COPY . .
RUN pip install .
```

Любое изменение README инвалидирует установку. Более управляемый подход копирует dependency metadata отдельно, а исходники — после. В учебном проекте простота также важна, поэтому оптимизацию нужно измерять по build log, а не превращать Dockerfile в нечитабельную головоломку.

5 Build-time и runtime

Build-time — создание image. Здесь допустимы tests, linters, компиляторы и исходные данные для учебной модели. Runtime — работа сервиса; здесь остаётся только необходимое для обслуживания запросов.

Смешение границ приводит к рискам:

- тестовые зависимости увеличивают поверхность атаки;
- исходные данные попадают в поставку;
- модель обучается при каждом запуске container;
- runtime требует доступ к сети и package index;
- ошибка становится зависимой от внешней доступности.

Обучение модели в build используется в ЛР6 как учебная гарантия воспроизводимости. В зрелом pipeline image и модель могут версионироваться отдельно, но release manifest обязан связывать их.

6 Многостадийная сборка

Первая стадия quality выполняет Ruff, pytest, contract validation, обучение и подготовку wheels. Вторая стадия runtime получает только wheels и модель.

Преимущества:

- failed test останавливает поставку;
- runtime не содержит tests и dev tools;
- граница между проверкой и эксплуатацией видна в коде;
- каждую стадию можно диагностировать отдельно.

Важно: наличие стадии tests ещё не означает gate. Финальная стадия должна зависеть от её успешного результата. Иначе builder может пропустить независимую стадию.

7 Base image

Base image является зависимостью. Общий тег вроде python:3.12-slim удобен для лабораторной, но может обновляться. --pull позволяет проверить свежую базу; production release обычно фиксирует проверенный digest и регулярно обновляет его отдельным процессом.

Не утверждайте, что самый маленький image автоматически самый безопасный. Важны происхождение, обновляемость, совместимость wheels, диагностируемость и реально необходимые пакеты.

8 Non-root runtime

USER задаёт пользователя для следующих инструкций и runtime-команды. Если сервису не нужны административные права, root нарушает принцип минимальных привилегий.

Проверка:

```
docker image inspect IMAGE --format '{{.Config.User}}'
docker run --rm --entrypoint id IMAGE
```

Non-root не является полной песочницей. Его дополняют cap_drop, no-new-privileges, read-only filesystem, ограничения ресурсов и безопасная конфигурация хоста.

9 Read-only filesystem и временные данные

Сервис предсказаний не должен менять код и модель. read_only: true делает случайную запись наблюдаемой ошибкой. Для действительно временных данных предоставляется небольшой tmpfs в /tmp.

Если приложение требует запись в произвольную директорию, сначала выясните причину. Добавлять широкий writable volume только ради устранения ошибки — значит скрыть нарушение контракта.

10 Секреты и конфигурация

Runtime configuration передаётся через environment или внешние механизмы, но значение секрета не должно попадать в Dockerfile, ARG, label, image history и отчёт. .env удобен локально, но исключается из Git и build context.

Разделяйте:

- несекретную конфигурацию — порт, путь модели, режим логов;
- секрет — token, пароль, ключ;
- артефакт — модель, contract, release manifest.

11 CMD и жизненный цикл процесса

Exec-форма CMD запускает процесс без лишней shell-обёртки и корректнее получает сигналы остановки. Container живёт, пока работает основной процесс PID 1. Фоновый запуск Uvicorn внутри shell-скрипта может нарушить обработку сигналов.

EXPOSE 8080 документирует порт image, но не публикует его на host. Публикацию задают docker run -p или Compose.

12 Health и readiness

Различайте:

- liveness/health — процесс способен отвечать;
- readiness — сервис загрузил модель и может выполнить полезную работу.

Контейнер с работающим Uvicorn, но отсутствующей моделью, жив, однако не готов. Поэтому healthcheck ЛР6 обращается к /readyz, а не только /healthz.

Healthcheck имеет start_period, interval, timeout и retries. Статус starting нормален во время инициализации. Нельзя заменять проверку готовности длинным фиксированным sleep.

13 Docker Compose

Compose фиксирует локальный способ запуска сервиса: build target, image tag, port mapping, environment, healthcheck и runtime restrictions. Современный compose.yaml не требует верхнеуровневого поля version.

Compose не превращает локальный стенд в production orchestrator. Его задача в курсе — дать одной команде воспроизводимый контур для ЛР6, ЛР7 и последующей наблюдаемости.

14 HTTP smoke test

Smoke test проверяет минимальный пользовательский путь через реальную сетевую границу:

1. дождаться /readyz до timeout;
2. вызвать /predict;
3. проверить status 200;
4. проверить форму и диапазоны ответа;
5. сохранить компактный отчёт.

Он не заменяет unit/API tests, но обнаруживает ошибки packaging, port mapping, модели и команды запуска.

15 Диагностика по слоям

Симптом	Первый источник	Возможная причина
build падает на pip	build log	dependency или сеть
build падает на pytest	test report	регрессия кода
container сразу завершился	docker compose logs	CMD или import error
health остаётся unhealthy	inspect health log	модель, URL или порт
503 на /readyz	application log	неверный MODEL_PATH
permission denied	user и mounts	запись вне разрешённого пути

Учите сначала локализовать фазу: build, create, start, readiness или request. Это сокращает область поиска.

16 Что не следует делать

- копировать весь репозиторий в runtime;
- использовать latest как доказательство версии;
- устанавливать пакеты при старте container;
- запускать сервис от root «для удобства»;
- встраивать token через ARG;
- считать EXPOSE публикацией порта;
- объявлять healthy процесс без загруженной модели;
- удалять секрет в позднем layer после его копирования;

- редактировать container вручную вместо пересборки image.

17 Индивидуальная ответственность

Участник должен владеть проверяемой частью границы: слоями сборки, безопасностью runtime, переносом модели, smoke test или review build context. На защите он показывает image ID, связанную строку Dockerfile/Compose и диагностирует назначенный отказ.

Для команды из пяти роли распределяются по независимым зонам. Для пары роли совмещаются, но у каждого остаётся собственный PR/review и отдельная операция диагностики.

18 Вопросы аудитории

1. Почему image не равен container?
2. Чем tag отличается от digest?
3. Может ли .dockerignore удалить секрет из существующего image?
4. Почему отдельная test stage может не быть gate?
5. Зачем runtime нужен non-root пользователь?
6. Почему /healthz недостаточно для ML-сервиса?
7. Что произойдёт при read_only: true и записи в /app?
8. Как связать image с commit и моделью?

19 Завершение

Контейнеризация завершена, когда команда может доказать происхождение image, успешность проверок, минимальность runtime, готовность модели и воспроизводимость HTTP-запроса. В ЛР7 эта граница получит формальный OpenAPI-контракт, типизированные ошибки и негативные сценарии клиента.

Чек-лист преподавателя

- Различены Dockerfile, image, container, tag и digest.
- Объяснены build context, layers, cache и .dockerignore.
- Quality stage является реальным gate runtime.
- Runtime не содержит tests, data и dev tools.
- Non-root дополнен read-only и security options.
- Health отделён от readiness модели.
- Smoke test проходит реальную HTTP-границу.
- Индивидуальная защита включает диагностику отказа.

Официальные источники

- Dockerfile reference: https://docs.docker.com/reference/dockerfile/
- Docker build best practices: https://docs.docker.com/build/building/best-practices/
- Multi-stage builds: https://docs.docker.com/get-started/docker-concepts/building-images/multi-stage-builds/
- Build context: https://docs.docker.com/build/concepts/context/
- Compose services: https://docs.docker.com/reference/compose-file/services/